Kingdom of Saudi Arabia

King Faisal University

College of Computer Sciences & Information Technology

Project Title:

Encrypt & Decrypt Using Playfair Cipher

Course Name:

Computer Security

Name	ID
Zainab Alhadhari	222405495
Anfal Alsuhayib	222430500
Atekah Aljafar	222427138
Maryam Alshabib	223050271

Supervised by :

Prof Mona Ali

Contents

Introduction:

This project is about building a simple encryption tool using the Playfair Cipher algorithm. In today's world, protecting information from unauthorized access is more important than ever. One common way to secure data is through encryption, which turns readable text (plaintext) into unreadable code (ciphertext). Later, decryption is used to turn it back into its original form. Many methods have been created for this, and each works in its way. One of the earliest and cleverest ones is the Playfair Cipher, developed in the 1800s by Charles Wheatstone. Unlike simple ciphers that replace one letter at a time, Playfair works with digraphs (letter pairs), making it harder to break using basic analysis. It uses a 5x5 letter grid based on a keyword and follows specific rules to encrypt each pair. The Playfair Cipher is a form of symmetric encryption, meaning the same key is used for both encryption and decryption.

In this report, we will explain how the Playfair Cipher works, describe how the program implements it from the user's perspective, and outline the steps of the encryption and decryption algorithm.

Description of Project Implementation:

In today's rapidly evolving digital landscape, maintaining the privacy of information has become a central concern. As cyber threats and data breaches continue to rise, it is essential to understand not only advanced encryption techniques but also the foundational methods that shaped modern cryptography. This project explores one such foundational method: the **Playfair Cipher**. Though simple by modern standards, it introduces key cryptographic principles that are still relevant today.

The core aim of the project is to **develop a functional program** that applies the Playfair Cipher to perform both encryption and decryption of alphabetic messages. This involves building a complete flow—from accepting user inputs to producing output securely and correctly according to the cipher's rules.

The implementation process begins with **user interaction**, where the user provides a keyword. This keyword is essential for constructing the **5x5 cipher matrix**. The matrix is built by removing duplicate characters from the keyword and then filling in the remaining unused letters of the alphabet, typically merging 'I' and 'J' to fit the grid. This matrix serves as the encryption and decryption key for the entire process.

Once the matrix is generated, the input text—either plaintext or ciphertext—is prepared. This involves several preprocessing steps:

- All characters are converted to uppercase.
- Spaces and non-alphabetic symbols are removed.
- For encryption, the text is split into **digraphs** (pairs of letters). If a digraph contains the same letter twice (e.g., "EE"), an 'X' is inserted between them to separate repeated letters.
- If the total number of characters is odd, an 'X' is appended to ensure complete pairs.

The heart of the implementation lies in the logic applied to each pair of letters:

- If both letters appear in the **same row** of the matrix, they are each replaced with the letter to their immediate right (wrapping around to the start of the row if needed).
- If both letters are in the **same column**, they are replaced by the letter immediately below them (again, wrapping to the top if necessary).
- If the letters form a **rectangle**, they are swapped across the corners of the rectangle—each letter is replaced by the letter in the same row but in the other's column.

The reverse of this process is used for decryption, with adjustments made to move **left** in rows and **up** in columns where necessary. This symmetrical nature of the Playfair Cipher makes it an effective teaching tool for understanding how symmetric encryption works.

Beyond its practical functionality, this implementation provides a hands-on opportunity to explore important cryptographic ideas such as key management, substitution, and pairwise analysis. These concepts serve as the basis for understanding how **modern encryption schemes** operate at a more advanced level, particularly in symmetric encryption systems.

By designing and implementing this classical cipher in code, we not only develop programming and algorithmic thinking skills but also gain insights into how simple logical rules can be combined to create a secure communication system. This helps reinforce the importance of even the most basic encryption methods in the broader context of data security.

Description of Algorithm :

Encryption

```
Procedure EncryptByPlayfairCipher(text, key)
  // Step 1: Prepare the key and text
  RemoveSpaces(key)
  ToLowerCase(key)
  RemoveSpaces(text)
  ToLowerCase(text)
// Step 2: Prepare a 5 x 5 matrix containing all letters without repetition.
 The letters are arranged according to the key and continue in alphabetical order
  keyT ← GenerateKeyTable(key)
// Step 3: Prepare the text where the text is divided into character pairs. If the pair contains two identical characters, an x is added between them. If the text is odd in length, an x is added at the end
  PrepareText(text)
  // Step4:Encrypt each pair of letters
  For i ← 0 to length(text) - 1 step 2 do
    a ← text[i]
    b ← text[i+1]
    (row1, col1, row2, col2) ← SearchPositions(keyT, a, b)
    If row1 == row2 then
// If the two characters are the same row: replace with the next character in the row
      text[i]   ← keyT[row1][(col1 + 1) mod 5]
      text[i+1] ← keyT[row2][(col2 + 1) mod 5]
    Else if col1 == col2 then
// If the two characters are in the same column, replace them with the next character in the column
      text[i]   ← keyT[(row1 + 1) mod 5][col1]
      text[i+1] ← keyT[(row2 + 1) mod 5][col2]
    Else
// Swap columns
      text[i]   ← keyT[row1][col2]
      text[i+1] ← keyT[row2][col1]
    EndIf
  EndFor
EndProcedure
```

Example of Encryption

Keyword: "SECURITY"

S E C U R

I/J T Y A B

D F G H K

L M N O P

Q V W X Z

Plaintext: "HELP"

1. **Preprocess Plaintext:**
    - **Split into digraphs: HE, LP.**
    - **No double letters or odd length → no padding needed.**
2. **Encryption:**
    - **"HE":**
        - **H (3,4) + E (1,2) → Rectangle → Swap with opposite corners:**
            - **H (3,4) → U (1,4)**
            - **E (1,2) → F (3,2)**
              → UF
    - **"LP":**
        - **L (4,1) + P (4,5) → Same Row → Shift right (wrap around):**
            - **L → M**
            - **P → Q**
              → MQ
3. **Ciphertext: "UF MQ" (or "UFMQ").**

Decryption

```
Procedure DecryptByPlayfairCipher(ciphertext, key)
  // Step 1: Prepare the key and text
  RemoveSpaces(key)
  ToLowerCase(key)
  RemoveSpaces(ciphertext)
  ToLowerCase(ciphertext)

  // Step 2: Generate 5x5 matrix (key table)
  keyT ← GenerateKeyTable(key)

  // Step 3: Decrypt each pair of letters
  For i ← 0 to length(ciphertext) - 1 step 2 do
    a ← ciphertext[i]
    b ← ciphertext[i+1]
    (row1, col1, row2, col2) ← SearchPositions(keyT, a, b)

    If row1 == row2 then
      // Same row: move left
      ciphertext[i]   ← keyT[row1][(col1 + 4) mod 5]
      ciphertext[i+1] ← keyT[row2][(col2 + 4) mod 5]
    Else if col1 == col2 then
      // Same column: move up
      ciphertext[i]   ← keyT[(row1 + 4) mod 5][col1]
      ciphertext[i+1] ← keyT[(row2 + 4) mod 5][col2]
    Else
      // Swap columns
      ciphertext[i]   ← keyT[row1][col2]
      ciphertext[i+1] ← keyT[row2][col1]
    EndIf
  EndFor
  Return ciphertext
EndProcedure
```

An Example of Decryption

1. **Ciphertext Digraphs: "UF", "MQ".**
2. **Decryption Rules:**
    - **"UF":**
        - **U (1,4) + F (3,2) → Rectangle → Swap back:**
            - **U → H (3,4)**
            - **F → E (1,2)**
              → HE
    - **"MQ":**
        - **M (4,2) + Q (5,1) → Same Row → Shift left (wrap around):**
            - **M → L**
            - **Q → P**
              → LP
3. **Decrypted Text: "HE LP" → "HELP".**

Conclusion:

In conclusion, the Playfair Cipher is a classic encryption technique that introduced the use of **digraphs** for greater security compared to simpler ciphers. While it may not be used widely in modern cryptography due to advancements in technology, it remains an excellent example of early cryptographic innovation. This report has explained the workings of the Playfair Cipher, detailed how the encryption and decryption processes occur, and demonstrated how the algorithm is implemented in the program from a user's perspective. Understanding such classical methods enhances our appreciation for modern cryptographic techniques and their evolution over time. Ultimately, the Playfair Cipher provides valuable insights into the balance between simplicity and security in cryptographic systems.

References:

- GeeksforGeeks. (n.d.). *Playfair Cipher with Examples*. GeeksforGeeks. Retrieved April 30, 2025, from https://www.geeksforgeeks.org/playfair-cipher-with-examples/
- Khan Academy. (n.d.). *Introduction to cryptography*. Khan Academy. Retrieved May 1, 2025, from https://www.khanacademy.org/computing/computer-science/cryptography
- Neso Academy. (2021, June 4). *Playfair Cipher (Part 1)* [Video]. YouTube. https://youtu.be/UURjVI5cw4g